# Техническое задание: Интеллектуальная система обнаружения мошеннических SMS-сообщений и подозрительных ссылок

# 1. Общие сведения

Итог проекта — работающее веб-приложение, которое принимает SMS или URL от пользователя, классифицирует их двумя обученными ML-моделями, рассчитывает уровень риска и объясняет результат. Проект выполняется за 18 последовательных заданий; каждое задание даёт проверяемую часть системы.

## 1.1. Наименование и шифр

- Полное наименование: «Интеллектуальная система обнаружения мошеннических SMS-сообщений и подозрительных ссылок на основе методов искусственного интеллекта и машинного обучения».
- Краткое наименование: **SMS Guard** (рабочее название, может быть изменено студентом).
- Тип работы: проект бакалавриата, уровень сложности — средний.

## 1.2. Назначение системы

Система предназначена для самостоятельной проверки пользователем входящих SMS и интернет-ссылок на признаки мошенничества, фишинга и социальной инженерии. Система носит **рекомендательный** характер: она оценивает вероятность угрозы, но не гарантирует безопасность сообщения или ссылки.

## 1.3. Актуальность

- Мошеннические SMS (смишинг) — один из самых массовых каналов кражи денег и учётных данных: злоумышленники выдают себя за банки, службы доставки, госорганы и операторов связи.
- Типовая схема атаки сочетает **текст с психологическим давлением** (срочность, угроза блокировки, выигрыш) и **ссылку** на поддельный сайт. Поэтому анализ только текста или только ссылки недостаточен.

- Рядовой пользователь не может быстро оценить структуру URL (поддомены, маскировка домена, сокращатели), а правила фильтрации у операторов не покрывают новые шаблоны сообщений.
- Методы машинного обучения позволяют выявлять статистические закономерности в тексте и структуре URL и обобщать их на новые сообщения.

## 1.4. Цель проекта

Разработать интеллектуальную систему обнаружения мошеннических SMS-сообщений и подозрительных ссылок с использованием методов машинного обучения и обработки естественного языка (NLP), реализованную в виде веб-приложения с пользовательским интерфейсом, базой данных и объяснением результатов.

## 1.5. Задачи проекта

1. Исследовать существующие методы обнаружения SMS-мошенничества и фишинговых URL.
2. Подготовить и обработать набор данных SMS-сообщений.
3. Реализовать предварительную обработку текстовых данных.
4. Разработать модель машинного обучения для классификации SMS.
5. Создать алгоритм извлечения признаков URL.
6. Разработать модель классификации интернет-ссылок.
7. Реализовать совместный анализ текста и URL.
8. Разработать механизм расчёта уровня риска.
9. Создать веб-интерфейс пользователя.
10. Реализовать регистрацию и авторизацию.
11. Создать базу данных для хранения результатов проверок.
12. Разработать панель мониторинга и статистики.
13. Реализовать историю проверок.
14. Провести тестирование системы.
15. Оценить качество разработанных моделей.
16. Подготовить демонстрационный сценарий работы приложения.

## 1.6. Категории результата

| Категория | Код | Смысл для пользователя |
|---|---|---|
| Безопасное | `SAFE` | Признаки мошенничества не обнаружены или незначительны |
| Подозрительное | `SUSPICIOUS` | Есть отдельные признаки угрозы; требуется осторожность |
| Мошенническое | `FRAUDULENT` | Совокупность признаков указывает на высокую вероятность мошенничества |

Для URL используются аналогичные категории: `SAFE`, `SUSPICIOUS`, `DANGEROUS` (потенциально опасная). Уровни риска: **низкий**, **средний**, **высокий**.

## 1.7. Термины и сокращения

| Термин | Определение |
|---|---|
| Смишинг (smishing) | Фишинг через SMS: попытка выманить деньги или данные с помощью текстового сообщения |
| Фишинг | Маскировка под доверенную организацию с целью получения учётных или платёжных данных |
| Социальная инженерия | Манипуляция поведением человека: срочность, страх, жадность, авторитет |
| URL | Унифицированный указатель ресурса (интернет-ссылка) |
| TF-IDF | Мера важности слова: частота в документе × обратная частота по корпусу |
| N-грамма | Последовательность из N подряд идущих токенов (слов или символов) |
| Recall (полнота) | Доля реально мошеннических объектов, найденных моделью |
| Precision (точность) | Доля верных срабатываний среди всех объектов, отмеченных как мошеннические |

| Термин | Определение |
|---|---|
| FN / FP | Ложноотрицательный (пропуск угрозы) / ложноположительный (ложная тревога) результат |
| Risk Score | Итоговый числовой показатель риска от 0 до 100 |
| Утечка данных (data leakage) | Попадание информации из тестовой выборки в процесс обучения |
| CSRF, XSS | Межсайтовая подделка запроса; межсайтовое выполнение скриптов |

# 2. Функциональные требования

Система состоит из семи функциональных модулей. Каждое требование имеет код (ФТ-x.y) и проверяется в отчёте о тестировании (задание 18).

## 2.1. Модуль регистрации и авторизации

| Код | Требование | Способ проверки |
|---|---|---|
| ФТ-1.1 | Регистрация по логину (3–30 символов, латиница, цифры, `_`), e-mail и паролю (не менее 8 символов, буквы и цифры) | Попытка регистрации с корректными и некорректными данными |
| ФТ-1.2 | Логин и e-mail уникальны; при повторе выводится понятная ошибка | Повторная регистрация |
| ФТ-1.3 | Авторизация по логину и паролю; после 5 неудачных попыток подряд — пауза не менее 60 с | Серия неверных входов |
| ФТ-1.4 | Пароли хранятся только как хеш с солью (PBKDF2, bcrypt или Argon2) | Просмотр таблицы пользователей в БД |
| ФТ-1.5 | Выход из системы уничтожает сессию | Повторный запрос защищённой страницы после выхода |

| Код | Требование | Способ проверки |
|---|---|---|
| ФТ-1.6 | Страницы проверки, истории, Dashboard и профиля доступны только авторизованному пользователю | Прямой переход по URL без входа → перенаправление на вход |
| ФТ-1.7 | Пользователь видит только свои проверки; запрос чужой записи возвращает 404 или 403 | Подмена id в адресе |
| ФТ-1.8 | Cookie сессии: `HttpOnly`, `SameSite=Lax`, `Secure` при HTTPS; все POST-формы защищены CSRF-токеном | Инструменты разработчика браузера |
| ФТ-1.9 | Роль «Администратор» видит агрегированную статистику по всем пользователям без текстов чужих SMS | Вход под администратором |

## 2.2. Модуль анализа SMS

| Код | Требование |
|---|---|
| ФТ-2.1 | Поле ввода SMS: от 1 до 1000 символов; пустой ввод и ввод только из пробелов отклоняются с сообщением |
| ФТ-2.2 | Определение языка текста (`ru`, `kk`, `en`, `other`) и вывод его в результате |
| ФТ-2.3 | Очистка и нормализация: нижний регистр, замена URL, телефонов, чисел и сумм на токены (`<URL>`, `<PHONE>`, `<NUM>`, `<MONEY>`), удаление лишних пробелов и управляющих символов |
| ФТ-2.4 | Извлечение TF-IDF-признаков той же векторизацией, что и при обучении (единый сохранённый pipeline) |

| Код | Требование |
|---|---|
| ФТ-2.5 | Поиск мошеннических выражений по словарю правил (не менее 6 групп: срочность, деньги/переводы, банк/карта, блокировка аккаунта, выигрыш/приз, запрос кода/пароля) |
| ФТ-2.6 | Классификация ML-моделью с выводом вероятности класса «мошенничество» `P_sms` от 0 до 1 |
| ФТ-2.7 | Если в тексте есть URL — автоматическая передача его в модуль проверки ссылок (до 3 ссылок на сообщение) |
| ФТ-2.8 | Для языка, не представленного в обучающих данных, выводится предупреждение о сниженной надёжности оценки |

**Ограничение по языкам.** Набор SMS Spam Collection содержит только англоязычные сообщения. Поддержка русского и казахского языков заявляется только при наличии размеченных данных на этих языках и отдельного теста (см. раздел 3.3). Без этого для `ru` и `kk` работают словарь правил и URL-анализ, а ML-оценка текста помечается как ненадёжная.

## 2.3. Модуль проверки ссылок

URL анализируется **только как строка**. Система не открывает ссылку, не загружает страницу и не переходит по редиректам.

| Код | Требование |
|---|---|
| ФТ-3.1 | Проверка формата: допустимые схемы `http`, `https` или URL без схемы (тогда добавляется `http://` для разбора); длина до 2048 символов |
| ФТ-3.2 | Извлечение не менее 20 числовых признаков (перечень — задание 9) |
| ФТ-3.3 | Обнаружение IP-адреса (IPv4/IPv6) вместо домена |
| ФТ-3.4 | Подсчёт поддоменов, точек, дефисов, цифр, спецсимволов ( `@` , `~` , `%` , `=` , `&` , `_` ) |

| Код | Требование |
|---|---|
| ФТ-3.5 | Проверка HTTPS, нестандартного порта (не 80/443), символа `@` в адресе, двойного `//` в пути |
| ФТ-3.6 | Распознавание URL-сокращателей по списку (не менее 15 сервисов) |
| ФТ-3.7 | Подозрительные ключевые слова ( `login` , `verify` , `secure` , `account` , `update` , `bank` , `bonus` , `free` и др.) |
| ФТ-3.8 | Признаки маскировки: Punycode ( `xn--` ), смешение алфавитов (кириллица + латиница), сходство с брендом из списка (расстояние Левенштейна ⩽ 2) |
| ФТ-3.9 | Классификация ML-моделью с вероятностью опасности `P_url` и категорией `SAFE` / `SUSPICIOUS` / `DANGEROUS` |
| ФТ-3.10 | Ссылки в результатах отображаются как неактивный текст (не кликабельны), домен выделяется |
| ФТ-3.11 | Необязательно: запрос к внешнему сервису репутации (например, Google Safe Browsing API) с таймаутом 3 с; при недоступности система работает только на собственной модели |

## 2.4. Интеллектуальный модуль

- **SMS-модель:** pipeline `TfidfVectorizer` + классификатор. Сравниваются не менее трёх алгоритмов: Multinomial Naive Bayes, Logistic Regression, Linear SVM (с калибровкой вероятностей `CalibratedClassifierCV` ) или Random Forest.
- **URL-модель:** числовые признаки + классификатор. Сравниваются не менее двух алгоритмов: Logistic Regression, Random Forest (допускается Gradient Boosting).
- Обе модели сохраняются в файлы ( `joblib` ) вместе с метаданными: версия, дата обучения, набор признаков, метрики на тесте, выбранный порог.
- Модели загружаются **один раз** при старте приложения, а не при каждом запросе.

- Объединение результатов двух моделей выполняет модуль оценки риска (2.5).

### 2.5. Модуль оценки риска

- Формирует **Risk Score** от 0 до 100 и уровень риска: низкий / средний / высокий.
- Учитывает: `P_sms`, максимальную `P_url` среди ссылок сообщения, число групп признаков социальной инженерии, совпадение нескольких подозрительных признаков.
- Пороги и веса хранятся в конфигурационном файле (`risk_config.yaml`), а не в коде, и подбираются на **валидационной** выборке.
- Для каждого результата формируется список причин (не менее одной) и рекомендация пользователю.
- В результате всегда отображается предупреждение: «Оценка носит вероятностный характер и не гарантирует безопасность».

### 2.6. Панель мониторинга (Dashboard)

Отображает для текущего пользователя (администратору — по всей системе):

- Общее число проверенных SMS и URL.
- Число безопасных, подозрительных и мошеннических результатов.
- Распределение по категориям (круговая или столбчатая диаграмма).
- Динамику проверок по дням за последние 30 дней (линейный график).
- Распределение уровней риска.
- Топ-5 сработавших признаков социальной инженерии.
- Таблицу из 5–10 последних проверок со ссылкой на подробный результат.

### 2.7. История проверок

Для каждой проверки сохраняются: дата и время, тип (`SMS` / `URL`), категория, уровень риска, Risk Score, вероятности моделей, краткое объяснение, версии моделей.

- Список с пагинацией по 20 записей, сортировка по дате (новые сверху).
- Фильтры: диапазон дат, тип проверки, категория, уровень риска.
- Просмотр подробного результата и удаление собственной записи.
- Текст SMS сохраняется **только при согласии** пользователя (флажок в профиле, по умолчанию выключен). Без согласия хранится маскированная версия: номера телефонов, карт и суммы заменены на `***`.
- Экспорт собственной истории в CSV (дополнительная функция).

# 3. Нефункциональные требования, технологии, данные и метрики

Нефункциональные требования заданы числами, чтобы их можно было проверить нагрузочным и функциональным тестом.

## 3.1. Нефункциональные требования

| Код | Группа | Требование | Проверка |
|---|---|---|---|
| НФТ-1 | Производительность | Анализ одного SMS (с извлечением до 3 URL) на сервере — не более 500 мс без учёта внешних API | Замер времени в логах, 100 запросов |
| НФТ-2 | Производительность | Загрузка Dashboard при 1000 записях истории — не более 2 с | Инструменты разработчика браузера |
| НФТ-3 | Производительность | Повторная отправка той же формы не создаёт дубликат записи (защита от двойного клика) | Двойное нажатие кнопки |
| НФТ-4 | Безопасность | Все запросы к БД — через ORM или параметризованные запросы; конкатенация SQL запрещена | Код-ревью, тест с `' OR 1=1 --` |
| НФТ-5 | Безопасность | Весь пользовательский ввод экранируется при выводе; `<script>` в SMS отображается как текст | Тест с `<script>alert(1)</script>` |
| НФТ-6 | Безопасность | Ограничение частоты API: не более 30 проверок в минуту на пользователя (ответ `429`) | Скрипт из 40 запросов |

| Код | Группа | Требование | Проверка |
|---|---|---|---|
| НФТ-7 | Безопасность | Секретный ключ, пароль БД и API-ключи — в `.env`, не в репозитории; `DEBUG=False` в демонстрационной сборке | Просмотр репозитория |
| НФТ-8 | Безопасность | Загрузка файлов (если реализована): только `.txt` / `.csv`, до 1 МБ, проверка содержимого, хранение вне веб-корня | Загрузка `.exe` и файла 5 МБ |
| НФТ-9 | Приватность | Тексты SMS не пишутся в лог-файлы; в логах — только id проверки и категория | Просмотр логов |
| НФТ-10 | Удобство | Адаптивная вёрстка: корректное отображение при ширине 360, 768 и 1366 px | Режим адаптивного дизайна браузера |
| НФТ-11 | Удобство | Каждая ошибка ввода сопровождается текстом на русском языке рядом с полем | Ввод некорректных данных |
| НФТ-12 | Надёжность | Пустой, слишком длинный, бинарный ввод и некорректный URL не вызывают ошибку 500 | Набор негативных тестов |
| НФТ-13 | Надёжность | Ошибки пишутся в `logs/app.log` с ротацией; уровень, время, модуль, сообщение | Просмотр файла |
| НФТ-14 | Надёжность | При отсутствии файла модели приложение стартует с понятным сообщением в логе, а страница проверки сообщает о недоступности анализа | Переименовать файл модели |

| Код | Группа | Требование | Проверка |
|---|---|---|---|
| НФТ-15 | Сопровождаемость | Покрытие модулей `ml/` и `risk/` модульными тестами — не менее 60% строк | `pytest --cov` |

## 3.2. Технологический стек

| Компонент | Обязательный выбор | Допустимая альтернатива |
|---|---|---|
| Язык | Python 3.11+ | — |
| Backend | Django 5.x + Django REST Framework | FastAPI + SQLAlchemy + Jinja2 |
| Frontend | HTML5, CSS3, JavaScript (ES6), Bootstrap 5 | — |
| Графики | Chart.js или Plotly.js | Matplotlib (только для отчёта) |
| ML | scikit-learn | — |
| Данные | pandas, NumPy | — |
| NLP | NLTK (стоп-слова, токенизация) | spaCy |
| Определение языка | `langdetect` или `lingua` | Правило по доле кириллицы и казахских букв (ә, ғ, қ, ң, ө, ұ, ү, һ, і) |
| Разбор URL | `urllib.parse`, `tldextract`, `ipaddress` | — |
| Сохранение моделей | `joblib` | `pickle` |
| БД | SQLite (разработка), PostgreSQL 15+ (демонстрация) | Только SQLite с обоснованием |
| Тестирование | `pytest`, `pytest-django`, `coverage` | `unittest` |
| Исследования | Jupyter Notebook | — |
| Контроль версий | Git + GitHub/GitLab | — |

## 3.3. Данные для обучения

| Набор | Объём (по описанию источника) | Метки | Назначение |
|---|---|---|---|
| UCI SMS Spam Collection | ≈ 5 574 SMS на английском | `ham` / `spam` | Обучение SMS-модели |
| PhiUSIIL Phishing URL Dataset (UCI) | ≈ 235 тыс. URL | `1` легитимный / `0` фишинг | Обучение URL-модели (основной вариант) |
| Malicious URLs Dataset (Kaggle) | ≈ 650 тыс. URL | `benign`, `phishing`, `defacement`, `malware` | Альтернатива для URL-модели |
| Собственный набор (создаёт студент) | не менее 100 искусственных SMS на русском и 50 на казахском | `safe` / `fraud` | Только тестирование многоязычности |

Объёмы указаны по описаниям наборов; студент обязан проверить их фактически и привести точные цифры в отчёте.

Обязательные действия для каждого набора:

1. Указать источник, ссылку, лицензию и дату загрузки.
2. Описать структуру: столбцы, типы, целевая переменная.
3. Проверить пропуски и некорректные записи.
4. Удалить точные дубликаты **до** разделения на выборки.
5. Рассчитать баланс классов.
6. Разделить данные стратифицированно: **70% обучение / 15% валидация / 15% тест**, `random_state=42`.
7. Сохранить индексы разбиения в файл, чтобы эксперимент был воспроизводим.

**Правила против утечки данных:**

- Векторизатор TF-IDF и масштабирование признаков обучаются только на обучающей выборке (внутри `Pipeline`).
- Подбор гиперпараметров и порогов выполняется на валидационной выборке или кросс-валидацией на обучающей.

- Тестовая выборка используется **один раз** — для финального отчёта о метриках.
- Для URL: один и тот же домен не должен попадать одновременно в обучение и тест (разбиение по группам `GroupShuffleSplit` по регистрируемому домену) — иначе модель «запоминает» домены.
- В URL-наборах с готовыми признаками (например, полученными при загрузке страницы) использовать только признаки, вычислимые из строки URL: система не загружает страницы.

**Многоязычность.** Казахскоязычные SMS в перечисленных открытых наборах отсутствуют. Студент обязан явно указать это ограничение в отчёте и в интерфейсе. Заявлять поддержку казахского или русского языка разрешается только после теста на собственном наборе с Recall мошеннического класса не ниже 0,80.

Допустимые способы расширения: ручная разметка искусственных сообщений, перевод части англоязычного набора с последующей ручной проверкой (с указанием этого в отчёте). Реальные SMS третьих лиц без их согласия использовать запрещено.

### 3.4. Метрики качества

- Accuracy, Precision, Recall, F1-score — для класса «мошенничество» отдельно и macro-усреднением.
- Confusion Matrix — в абсолютных числах.
- ROC-AUC и PR-AUC — для моделей с вероятностной оценкой.
- Время предсказания одного объекта (мс).

**Приоритет Recall.** Пропуск мошеннического сообщения опаснее ложной тревоги. Итоговая модель выбирается по максимуму F2-score (Recall весит вдвое больше Precision) при условии Precision ⩾ 0,85.

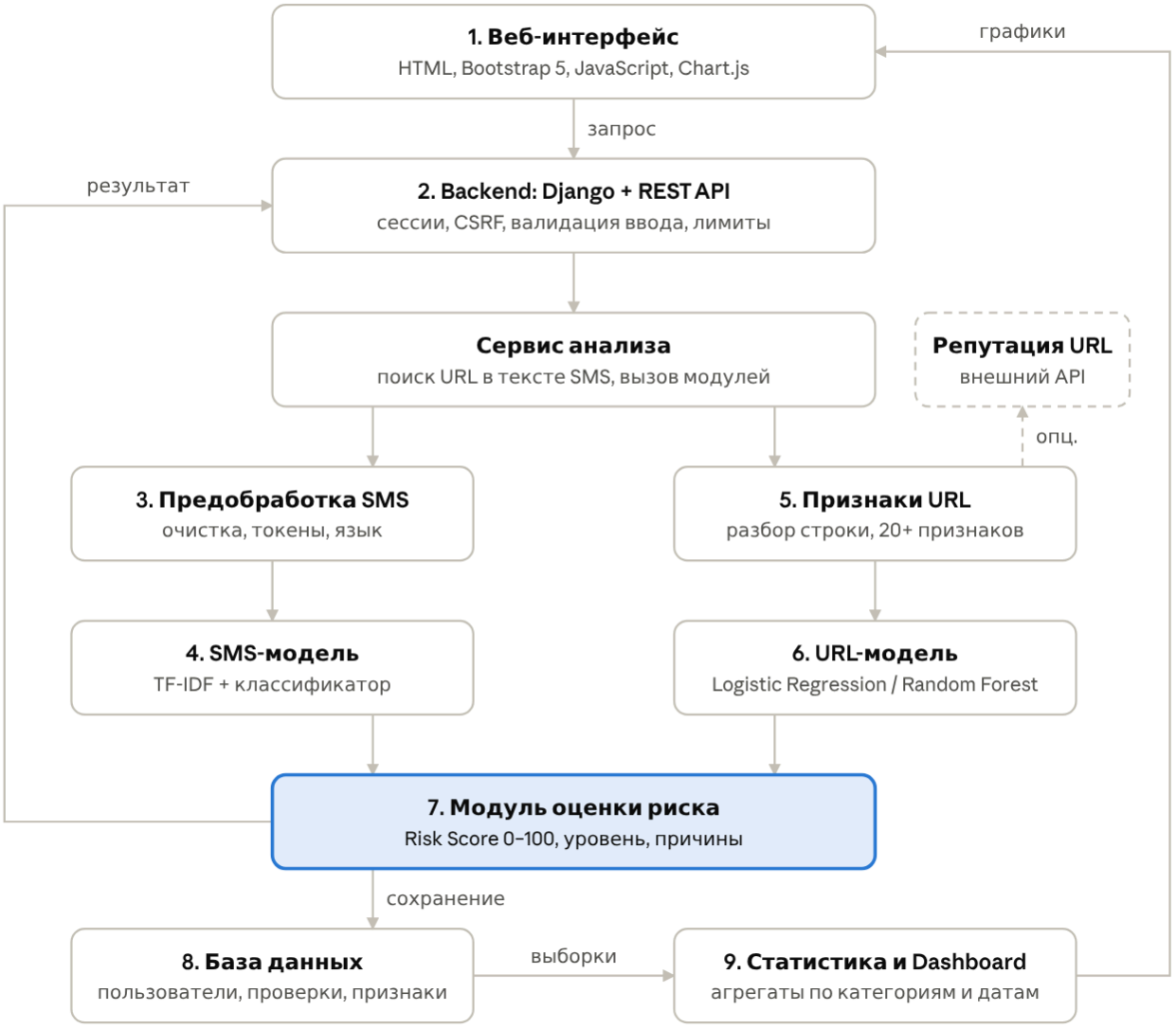
| Модель | Минимальные требования на тестовой выборке |
|---|---|
| SMS-классификатор | Recall (spam) ⩾ 0,90; Precision (spam) ⩾ 0,90; F1 ⩾ 0,90 |
| URL-классификатор | Recall (phishing) ⩾ 0,85; Precision ⩾ 0,85; ROC-AUC ⩾ 0,90 |

Если требования не достигнуты, студент приводит анализ причин (ошибочные примеры, баланс классов, признаки) — это не снижает оценку при корректной методике.

## 4. Архитектура системы

Система строится как монолитное веб-приложение Django с выделенными модулями ML и оценки риска. Модели загружаются в память один раз при старте и вызываются сервисом анализа.

**Анализ SMS и URL идёт двумя ветвями и сходится в модуле риска**




архитектура системы · 9 компонентов, две ветви анализа

Сервис анализа направляет текст SMS в левую ветвь, а найденные в нём ссылки — в правую; модуль риска объединяет обе оценки, результат сохраняется в БД и попадает в статистику. Внешний сервис репутации необязателен и не заменяет URL-модель.

## 4.1. Поток данных при проверке SMS

1. Пользователь вводит текст и нажимает «Проверить сообщение»; браузер отправляет POST с CSRF-токеном.
2. Backend проверяет авторизацию, лимит запросов и длину текста.
3. Сервис анализа извлекает URL регулярным выражением и определяет язык.
4. Текст нормализуется (URL заменяются токеном `<URL>`) и передаётся в SMS-pipeline → `P_sms`.
5. Словарь правил возвращает список сработавших групп социальной инженерии.
6. Каждый URL проходит извлечение признаков и URL-модель → `P_url_i`.
7. Модуль риска рассчитывает Risk Score, уровень, категорию, причины и рекомендацию.
8. Результат сохраняется в БД (с учётом согласия на хранение текста) и возвращается пользователю.

## 4.2. Структура репозитория

```
sms_guard/
├── config/                # настройки Django, urls.py, wsgi.py
├── apps/
│   ├── accounts/          # регистрация, вход, профиль
│   ├── analysis/          # сервис анализа, views, API, модели БД проверок
│   └── dashboard/         # статистика и графики
├── ml/
│   ├── preprocessing.py   # очистка и нормализация SMS
│   ├── url_features.py    # извлечение признаков URL
│   ├── rules.py           # словарь социальной инженерии
│   ├── risk.py            # расчёт Risk Score
│   ├── predictor.py       # загрузка моделей и предсказание
│   └── artifacts/         # sms_model.joblib, url_model.joblib,
metadata.json
├── notebooks/             # 01_eda_sms.ipynb ... 05_url_models.ipynb
├── data/
│   ├── raw/               # исходные наборы (в .gitignore, если большие)
│   └── processed/         # очищенные данные, индексы разбиения
├── templates/             # HTML-шаблоны
├── static/                # CSS, JS, изображения
├── tests/                 # pytest
├── docs/                  # отчёты, диаграммы
```

```
├── risk_config.yaml
├── requirements.txt
├── .env.example
└── README.md
```

## 4.3. Модель данных (основа ER-диаграммы)

| Таблица | Ключевые поля | Связи |
|---|---|---|
| `users` (встроенная модель Django) | id, username, email, password (хеш), date_joined, is_staff | 1→N `checks` |
| `user_profile` | user_id, store_sms_text (bool, по умолчанию false), preferred_language | 1→1 `users` |
| `checks` | id, user_id, check_type (`SMS` / `URL`), input_text (nullable), input_masked, language, category, risk_level, risk_score (0–100), p_sms (nullable), explanation, recommendation, sms_model_version, url_model_version, created_at | N→1 `users`; 1→N `url_results`, `detected_features` |
| `url_results` | id, check_id, url, domain, p_url, category, features_json | N→1 `checks` |
| `detected_features` | id, check_id, code (например, `URGENCY`), title, weight | N→1 `checks` |
| `model_versions` | id, model_type (`sms` / `url`), version, algorithm, metrics_json, threshold, trained_at, is_active | справочная |

Индексы: `checks(user_id, created_at)`, `checks(category)`. Каскадное удаление `url_results` и `detected_features` при удалении проверки.

## 4.4. REST API

| Метод | Эндпоинт | Назначение | Ответ |
|---|---|---|---|
| POST | `/api/v1/auth/register/` | Регистрация | `201`, данные пользователя |
| POST | `/api/v1/auth/login/` | Вход (сессия или токен) | `200` / `401` |
| POST | `/api/v1/auth/logout/` | Выход | `204` |
| POST | `/api/v1/check/sms/` | Анализ SMS: `{"text": "..."}` | `200`, объект результата |
| POST | `/api/v1/check/url/` | Анализ URL: `{"url": "..."}` | `200`, объект результата |
| GET | `/api/v1/checks/` | История с фильтрами `date_from`, `date_to`, `type`, `category`, `risk_level`, `page` | `200`, список |
| GET | `/api/v1/checks/{id}/` | Подробный результат | `200` / `404` |
| DELETE | `/api/v1/checks/{id}/` | Удаление своей записи | `204` / `404` |
| GET | `/api/v1/stats/summary/` | Данные для Dashboard | `200` |
| GET | `/api/v1/health/` | Статус приложения и загруженных моделей | `200` |

Пример ответа `/api/v1/check/sms/`:

```
{
  "id": 128,
  "type": "SMS",
  "language": "en",
  "category": "FRAUDULENT",
  "risk_level": "HIGH",
  "risk_score": 87,
  "p_sms": 0.94,
  "urls": [{"url": "http://secure-bank-verify.example", "p_url": 0.81,
"category": "DANGEROUS"}],
  "features": ["URGENCY", "ACCOUNT_BLOCK", "LINK_REQUEST"],
  "explanation": ["Сообщение требует срочного действия", "..."],
  "recommendation": "Не переходите по ссылке и не вводите данные карты",
  "disclaimer": "Оценка носит вероятностный характер",
  "model_versions": {"sms": "1.0.0", "url": "1.0.0"},
  "created_at": "2026-10-31T10:15:00+05:00"
}
```


Ошибки возвращаются в едином формате: `{"error": {"code": "VALIDATION_ERROR", "message": "Текст SMS не может быть пустым"}}` с кодами `400`, `401`, `403`, `404`, `429`, `503` (модель не загружена).

## 5. Практические задания: этап 1 — подготовка и данные SMS (задания 1–6)

Задания выполняются строго последовательно: результат каждого задания является входом следующего. Каждое задание завершается коммитом в Git с сообщением вида `task-05: EDA SMS dataset`.

### Задание 1. Анализ предметной области и проектирование системы

**Цель.** Сформировать понимание проблемы SMS-мошенничества и зафиксировать требования к системе до начала разработки.

**Описание.** Студент изучает типовые схемы смишинга и фишинговых ссылок, существующие решения и формулирует сценарии использования системы.

**Функциональные требования:**

1. Описать не менее 6 категорий угроз: поддельный банк, «блокировка карты», выигрыш/приз, служба доставки, «родственник в беде», запрос кода из SMS, поддельные госуслуги/штрафы, инвестиционное мошенничество.

2. Для каждой категории привести один искусственный пример SMS и перечислить признаки социальной инженерии (срочность, страх, авторитет, выгода, секретность).
3. Проанализировать не менее 3 аналогов (встроенные фильтры спама смартфонов, Google Safe Browsing, VirusTotal, антиспам операторов связи): назначение, сильные стороны, ограничения.
4. Определить целевую аудиторию и не менее 2 ролей: «Пользователь», «Администратор».
5. Построить диаграмму вариантов использования (Use Case) — не менее 8 прецедентов.
6. Составить таблицу требований (ФТ и НФТ) на основе разделов 2–3 настоящего ТЗ с указанием приоритета (MoSCoW: Must / Should / Could).

**Ожидаемый результат:** документ `docs/01_domain_analysis.md` (5–8 страниц), диаграмма Use Case (PlantUML, draw.io или аналог), таблица требований.

**Критерии проверки:**

- [ ] Описано ⩾ 6 категорий угроз с примерами и признаками.
- [ ] Сравнение ⩾ 3 аналогов оформлено таблицей.
- [ ] Use Case содержит ⩾ 8 прецедентов и обе роли.
- [ ] Каждое требование имеет код и приоритет.

## Задание 2. Подготовка структуры проекта

**Цель.** Создать воспроизводимую рабочую среду и каркас приложения.

**Описание.** Студент создаёт репозиторий, виртуальное окружение и Django-проект со структурой из раздела 4.2.

**Функциональные требования:**

1. Git-репозиторий с `.gitignore` (исключены `.venv`, `.env`, `__pycache__`, большие файлы данных).
2. Виртуальное окружение (`venv` или `poetry`), Python 3.11+.
3. `requirements.txt` с **зафиксированными** версиями библиотек (`pandas==2.2.*` и т. п.).
4. Django-проект с приложениями `accounts`, `analysis`, `dashboard` и пакетом `ml/`.
5. Настройки разделены через переменные окружения (`django-environ` или `python-dotenv`); файл `.env.example` с перечнем переменных без значений секретов.

6. Настроено логирование в `logs/app.log` (ротация, 5 файлов по 1 МБ).
7. Базовый шаблон `base.html` с Bootstrap 5 и навигацией; главная страница-заглушка открывается.
8. Эндпоинт `/api/v1/health/` возвращает `{"status": "ok"}`.
9. `README.md` с инструкцией запуска из 5–7 команд.

**Ожидаемый результат:** проект запускается командой `python manage.py runserver`, отвечает главная страница и health-эндпоинт.

**Критерии проверки:**

- [ ] Чистое клонирование + выполнение README приводит к работающему проекту.
- [ ] В репозитории нет секретов и `.env`.
- [ ] Версии всех библиотек зафиксированы.
- [ ] `python manage.py check` выполняется без ошибок.

## Задание 3. Сбор и загрузка набора данных SMS

**Цель.** Получить размеченный набор SMS для обучения модели.

**Описание.** Студент загружает UCI SMS Spam Collection, изучает формат и приводит его к единой структуре.

**Функциональные требования:**

1. Загрузить набор из официального источника; сохранить в `data/raw/` без изменений.
2. Написать скрипт `ml/data/load_sms.py`, который читает файл (разделитель — табуляция, кодировка проверяется явно) и возвращает DataFrame со столбцами `id`, `text`, `label` (`0` — ham/безопасное, `1` — spam/мошенничество).
3. Вывести: число строк, типы столбцов, 5 примеров каждого класса, распределение классов.
4. Заполнить «паспорт набора данных»: источник, ссылка, лицензия, дата загрузки, язык, объём, классы, известные ограничения (только английский язык, спам ≠ мошенничество в строгом смысле).
5. Подготовить собственный тестовый набор `data/raw/custom_ru_kk.csv`: не менее 100 искусственных SMS на русском и 50 на казахском (≈ 50% мошеннических), с указанием автора разметки.
6. Сохранить результат в `data/processed/sms_raw.csv` (UTF-8).

**Ожидаемый результат:** файл `sms_raw.csv`, паспорт набора данных, собственный многоязычный тестовый набор.

**Критерии проверки:**

- [ ] Скрипт загрузки воспроизводим и не зависит от абсолютных путей.
- [ ] Метки приведены к 0/1, число строк совпадает с исходным файлом.
- [ ] Паспорт содержит все 8 пунктов.
- [ ] Собственный набор не содержит реальных номеров телефонов, карт и персональных данных.

## Задание 4. Предварительная обработка SMS

**Цель.** Получить чистый текст, пригодный для векторизации, и единую функцию обработки для обучения и для работы приложения.

**Описание.** Студент реализует модуль `ml/preprocessing.py`. Одна и та же функция `normalize_sms(text) -> str` используется и в ноутбуке, и в веб-приложении.

**Функциональные требования:**

1. Сохранить исходный текст в столбце `text_raw`, обработанный — в `text_clean`.
2. Приведение к нижнему регистру; Unicode-нормализация NFKC.
3. Замена сущностей токенами: URL → `<url>`, e-mail → `<email>`, телефон → `<phone>`, сумма с валютой (`$`, `£`, `₸`, `руб`, `тг`) → `<money>`, прочие числа → `<num>`.
4. Удаление управляющих символов и повторяющихся пробелов; сохранение `!` и `?` как признаков давления (заменить на `!`, `?`).
5. Удаление стоп-слов — **как опция**; эффект проверяется в задании 8.
6. Обработка пропусков: строки с пустым текстом удаляются и учитываются в отчёте.
7. Удаление дубликатов по `text_clean`; число удалённых записей фиксируется по классам.
8. Не менее 8 модульных тестов `tests/test_preprocessing.py` (пустая строка, только URL, смешанный регистр, кириллица, казахские буквы, суммы, телефоны, эмодзи).

**Ожидаемый результат:** `data/processed/sms_clean.csv`, модуль предобработки, отчёт о преобразованиях (таблица «шаг — число затронутых записей»).

**Критерии проверки:**

- [ ] `normalize_sms` — чистая функция без побочных эффектов.
- [ ] Все тесты проходят.
- [ ] Казахские буквы (ә, ғ, қ, ң, ө, ұ, ү, h, i) не удаляются.
- [ ] Отчёт содержит число дубликатов по каждому классу.

## Задание 5. Исследовательский анализ SMS-данных (EDA)

**Цель.** Понять структуру данных и сформулировать гипотезы для построения признаков.

**Описание.** Анализ выполняется в `notebooks/01_eda_sms.ipynb` на очищенных данных.

**Функциональные требования:**

1. Распределение классов (столбчатая диаграмма с подписями количества и процента).
2. Распределение длины сообщений в символах и словах по классам (гистограмма или boxplot).
3. Топ-20 слов и топ-20 биграмм для каждого класса (без стоп-слов).
4. Доля сообщений с `<url>`, `<phone>`, `<money>`, `!` по классам.
5. Облако слов — по желанию, не заменяет пункт 3.
6. Вывод: не менее 5 сформулированных наблюдений (например, «мошеннические SMS в среднем длиннее»), каждое подтверждено числом.

**Ожидаемый результат:** ноутбук и не менее трёх графиков, сохранённых в `docs/figures/` (PNG, подписи осей на русском).

**Критерии проверки:**

- [ ] ⩾ 3 графика с заголовками и подписанными осями.
- [ ] ⩾ 5 выводов с числами.
- [ ] Указан дисбаланс классов и как он будет учтён (стратификация, `class_weight`, выбор метрик).
- [ ] Анализ выполнен только на обучающей выборке или на всём наборе **до** обучения без подбора параметров по тесту.

## Задание 6. Извлечение признаков из текста

**Цель.** Преобразовать SMS в числовое представление для ML-моделей.

**Описание.** Студент разделяет данные на выборки и строит векторизацию TF-IDF с N-граммами.

**Функциональные требования:**

1. Стратифицированное разбиение 70/15/15 с `random_state=42`; индексы сохранить в `data/processed/sms_split.json`.
2. `TfidfVectorizer` на словах: `ngram_range=(1, 2)`, `min_df=2`, `max_df=0.95`, `sublinear_tf=True`.
3. Дополнительно — `TfidfVectorizer` на символах: `analyzer='char_wb'`, `ngram_range=(3, 5)` (устойчив к опечаткам и намеренным искажениям: `fr33`, `b@nk`).
4. Объединение через `FeatureUnion` или `ColumnTransformer`.
5. Не менее 5 ручных признаков: длина, доля заглавных букв в исходном тексте, число `!`, наличие `<url>`, наличие `<money>`, число сработавших групп словаря.
6. Векторизатор обучается **только** на обучающей выборке (`fit` на train, `transform` на val/test).
7. Вывести размерность матриц признаков и топ-15 признаков с наибольшим средним TF-IDF для каждого класса.
8. Письменно объяснить назначение TF-IDF, N-грамм и символьных N-грамм (0,5–1 страница).

**Ожидаемый результат:** разреженные матрицы признаков `X_train`, `X_val`, `X_test`, объяснение признаков, код векторизации в виде функции, возвращающей `Pipeline`.

**Критерии проверки:**

- [ ] Нет вызова `fit` на валидационных или тестовых данных.
- [ ] Размерности матриц согласованы с размерами выборок.
- [ ] Векторизация реализована внутри `Pipeline` и сохраняется вместе с моделью.

## 6. Практические задания: этап 2 — модели и совместный анализ (задания 7–12)

На этом этапе создаются две обученные модели и модуль, объединяющий их результаты. Все эксперименты фиксируются в ноутбуках `notebooks/02-05`, итоговый код — в пакете `ml/`.

## Задание 7. Обучение первой ML-модели

**Цель.** Получить базовую (baseline) модель классификации SMS и отработать полный цикл «обучение → оценка → сохранение».

**Описание.** Студент обучает Multinomial Naive Bayes или Logistic Regression в составе `Pipeline` с векторизатором из задания 6.

**Функциональные требования:**

1. Обучить модель на обучающей выборке; для Logistic Regression задать `class_weight='balanced'`, `max_iter=1000`.
2. Получить предсказания и вероятности (`predict_proba`) на валидационной выборке.
3. Рассчитать Accuracy, Precision, Recall, F1 для класса «мошенничество», построить Confusion Matrix.
4. Вывести 10 ошибочно классифицированных сообщений (5 FN и 5 FP) и кратко прокомментировать причины.
5. Сохранить pipeline в `ml/artifacts/sms_baseline.joblib`; загрузить его в отдельной ячейке и убедиться, что предсказания совпадают.
6. Написать функцию `predict_sms(text) -> dict` (`label`, `probability`), использующую сохранённый pipeline и `normalize_sms`.

**Ожидаемый результат:** обученная baseline-модель, метрики на валидации, функция предсказания.

**Критерии проверки:**

- [ ] Модель загружается из файла и работает на новом тексте, введённом вручную.
- [ ] Метрики рассчитаны на валидационной, а не на обучающей выборке.
- [ ] Разобраны примеры ошибок.

## Задание 8. Сравнение алгоритмов классификации SMS

**Цель.** Выбрать обоснованно лучшую модель SMS с учётом приоритета Recall.

**Описание.** Сравниваются не менее трёх алгоритмов на одинаковых выборках и признаках.

**Функциональные требования:**

1. Обучить: Multinomial Naive Bayes, Logistic Regression, Linear SVM (`LinearSVC` + `CalibratedClassifierCV` для вероятностей), Random Forest (по желанию — четвёртый).

2. Подбор гиперпараметров: `GridSearchCV` или `RandomizedSearchCV` с `StratifiedKFold(n_splits=5)` на обучающей выборке, метрика — F1 или F2 для класса «мошенничество». Минимальная сетка: `alpha` для NB, `C` для LR и SVM, `n_estimators`, `max_depth` для RF.
3. Сравнить варианты признаков: только слова; слова + символьные N-граммы; со стоп-словами и без.
4. Сводная таблица на валидации: Accuracy, Precision, Recall, F1, F2, ROC-AUC, PR-AUC, время обучения, время предсказания одного SMS.
5. ROC- и PR-кривые всех моделей на одном графике.
6. Подбор порога классификации на валидации: минимальный порог, при котором Precision ⩾ 0,85, с максимальным Recall. Порог записывается в метаданные.
7. Выбрать итоговую модель, обосновать выбор (качество, скорость, интерпретируемость).
8. **Однократно** оценить выбранную модель на тестовой выборке; переобучить на train + val и сохранить как `ml/artifacts/sms_model.joblib` с `sms_model_meta.json` (алгоритм, параметры, порог, метрики на тесте, дата, версия `1.0.0`).
9. Проверить модель на собственном русско-казахском наборе и привести метрики отдельно по языкам.

**Ожидаемый результат:** сравнительная таблица, графики, итоговая модель SMS с метаданными, вывод по многоязычности.

**Критерии проверки:**

- [ ] Сравнено ⩾ 3 алгоритмов на одинаковом разбиении.
- [ ] Тестовая выборка не использовалась для подбора параметров и порога.
- [ ] Итоговая модель удовлетворяет требованиям раздела 3.4 или приведён анализ причин.
- [ ] Метрики на русском и казахском наборах приведены честно, без экстраполяции.

## Задание 9. Разработка модуля анализа URL

**Цель.** Реализовать извлечение структурных признаков ссылки без обращения к сети.

**Описание.** Студент создаёт модуль `ml/url_features.py` с функцией `extract_url_features(url: str) -> dict`. Функция используется и для подготовки датасета, и в приложении.

**Функциональные требования.** Функция возвращает не менее 20 признаков:

| № | Признак | Тип |
|---|---|---|
| 1 | Длина URL | число |
| 2 | Длина домена (host) | число |
| 3 | Длина пути | число |
| 4 | Число точек в URL | число |
| 5 | Число поддоменов (по `tldextract`) | число |
| 6 | Число дефисов в домене | число |
| 7 | Число цифр в URL | число |
| 8 | Доля цифр в домене | доля |
| 9 | Число спецсимволов ( `@ ~ % = & _ ?` ) | число |
| 10 | IP-адрес вместо домена (IPv4/IPv6) | 0/1 |
| 11 | Схема HTTPS | 0/1 |
| 12 | Нестандартный порт | 0/1 |
| 13 | Символ `@` в URL | 0/1 |
| 14 | Двойной `//` в пути | 0/1 |
| 15 | URL-сокращатель (список ≥ 15: bit.ly, tinyurl.com, goo.gl, t.co, is.gd, cutt.ly, ow.ly, clck.ru и др.) | 0/1 |
| 16 | Число подозрительных слов (login, verify, secure, account, update, bank, confirm, password, bonus, free, prize, kaspi, halyk и др.) | число |
| 17 | Punycode ( `xn--` ) в домене | 0/1 |
| 18 | Смешение алфавитов в домене | 0/1 |
| 19 | Сходство с брендом (Левенштейн ⩽ 2 к списку ⩾ 20 доменов банков и сервисов) при несовпадении домена | 0/1 |
| 20 | Энтропия Шеннона символов домена | число |

| № | Признак | Тип |
|---|---|---|
| 21 | Подозрительная доменная зона (список: `.xyz`, `.top`, `.click`, `.zip` и др., обосновать источник) | 0/1 |
| 22 | Число параметров запроса | число |

Дополнительно:

1. Функция `validate_url(url)` проверяет формат и возвращает понятную ошибку; URL без схемы дополняется `http://` только для разбора.
2. Функция `explain_url(features) -> list[str]` формирует человекочитаемые причины (например, «Используется IP-адрес вместо доменного имени»).
3. Запрещены сетевые запросы: `requests`, `socket`, DNS-разрешение внутри модуля.
4. Не менее 12 модульных тестов на синтетических URL (используйте зарезервированные домены `example.com`, `example.org`, `.test`).

**Ожидаемый результат:** модуль признаков URL, функции валидации и объяснения, тесты.

**Критерии проверки:**

- [ ] Возвращается словарь с фиксированным набором ключей одинакового порядка.
- [ ] Некорректный ввод не вызывает необработанного исключения.
- [ ] В модуле нет сетевых вызовов (проверяется код-ревью).
- [ ] Все тесты проходят.

## Задание 10. Подготовка датасета URL

**Цель.** Получить чистый размеченный набор URL и матрицу признаков для обучения.

**Описание.** Студент выбирает открытый набор из раздела 3.3 и строит признаки функцией из задания 9.

**Функциональные требования:**

1. Загрузить набор, заполнить паспорт (как в задании 3).
2. Привести метки к бинарному виду: `0` — безопасный, `1` — фишинг/вредоносный. Для набора с 4 классами обосновать, какие классы объединяются.

3. Удалить пустые, некорректные и дублирующиеся URL; нормализовать регистр домена.
4. При большом объёме допускается стратифицированная подвыборка 50–100 тыс. записей с сохранением баланса.
5. Проверить и при необходимости устранить **артефакты разметки**: например, если все безопасные URL имеют схему `https://www.`, а фишинговые — без схемы, модель выучит не тот признак. Привести URL к единому виду или исключить такие признаки и описать решение.
6. Разбиение 70/15/15 по группам регистрируемого домена (`GroupShuffleSplit`).
7. Построить признаки, сохранить в `data/processed/url_features.parquet`.
8. EDA: баланс классов и сравнение 6 ключевых признаков по классам (boxplot или таблица средних).

**Ожидаемый результат:** очищенный датасет URL, матрица признаков, описание структуры и предпринятых очисток.

**Критерии проверки:**

- [ ] Нет пересечения доменов между train и test.
- [ ] Описаны и устранены артефакты разметки.
- [ ] Признаки строятся той же функцией, что используется в приложении.

## Задание 11. Обучение модели обнаружения опасных ссылок

**Цель.** Получить сохранённую модель классификации URL.

**Описание.** Сравниваются не менее двух алгоритмов на признаках из задания 10.

**Функциональные требования:**

1. Обучить Logistic Regression (со `StandardScaler` в pipeline) и Random Forest; по желанию — Gradient Boosting (`HistGradientBoostingClassifier`).
2. Подбор гиперпараметров кросс-валидацией на обучающей выборке.
3. Таблица метрик на валидации (как в задании 8), ROC-кривые.
4. Важность признаков: коэффициенты LR и `feature_importances_` или permutation importance для RF; график топ-10.
5. Подбор порогов на валидации для трёх категорий URL: `SAFE` ($P < t1$), `SUSPICIOUS` ($t1 \leqslant P < t2$), `DANGEROUS` ($P \geqslant t2$); значения t1 и t2 обосновать.
6. Однократная оценка на тесте; сохранить `ml/artifacts/url_model.joblib` и `url_model_meta.json`.

7. Проверить модель на 20 искусственных URL (10 безопасных с реальными доменами известных организаций, 10 подозрительных на зарезервированных доменах) и прокомментировать результаты.

**Ожидаемый результат:** сохранённая URL-модель, таблица метрик, анализ важности признаков.

**Критерии проверки:**

- [ ] Сравнено ⩾ 2 алгоритмов.
- [ ] Пороги t1, t2 подобраны на валидации и хранятся в метаданных.
- [ ] Модель удовлетворяет требованиям раздела 3.4 или приведён анализ причин.

## Задание 12. Совместный интеллектуальный анализ SMS и URL

**Цель.** Объединить две модели в единый сервис анализа сообщения.

**Описание.** Студент реализует `ml/predictor.py` (загрузка моделей один раз, класс-синглтон) и `apps/analysis/services.py` с функцией `analyze_sms(text) -> AnalysisResult`.

**Функциональные требования:**

1. Извлечение URL из текста регулярным выражением, учитывающим: схемы `http/https`, адреса без схемы (`www.`, `bit.ly/...`), домены с кириллицей; не более 3 URL на сообщение.
2. Если URL найдены: текст анализируется SMS-моделью, каждый URL — URL-моделью; в результат попадают все `P_url` и максимум `P_url_max`.
3. Если URL нет: выполняется только текстовый анализ, поле `urls` пустое, `P_url_max = null`.
4. Словарь социальной инженерии `ml/rules.py`: не менее 6 групп, по 10+ выражений на русском, казахском и английском в каждой. Группы: `URGENCY` (срочно, немедленно, в течение 24 часов, тез арада), `MONEY_TRANSFER`, `BANK_CARD`, `ACCOUNT_BLOCK`, `PRIZE`, `CODE_REQUEST` (сообщите код, назовите код из SMS), по желанию `AUTHORITY`, `SECRECY`.
5. Функция `analyze_url(url)` для отдельной проверки ссылки без текста.
6. Возвращаемая структура `AnalysisResult` (dataclass): все поля из примера ответа API (раздел 4.4).
7. Интеграционные тесты: SMS без URL, SMS с одним URL, SMS с тремя URL, только URL.

**Ожидаемый результат:** единый модуль анализа, формирующий итоговый результат проверки сообщения.

**Критерии проверки:**

- [ ] Модели загружаются один раз за жизнь процесса (проверка через лог).
- [ ] При отсутствии URL система не падает и не выдаёт `P_url`.
- [ ] Ссылки из текста анализируются по отдельности и отображаются в результате.
- [ ] Ни одна ссылка не открывается сетевым запросом.

## 7. Практические задания: этап 3 — продукт (задания 13–18)

На этом этапе модели превращаются в веб-продукт: оценка риска, API, интерфейс, хранение, статистика и тестирование.

### Задание 13. Разработка алгоритма оценки риска

**Цель.** Превратить вероятности моделей и признаки социальной инженерии в понятный уровень риска с объяснением.

**Описание.** Студент реализует `ml/risk.py` с функцией `compute_risk(p_sms, p_url_max, se_groups, url_flags) -> RiskResult`. Базовая формула (веса — начальные, уточняются на валидации):

```
\text{RiskScore} = 100 \cdot \frac{w_1 P_{sms} + w_2 P_{url,max} + w_3 S_{se}}{w_1 + w_2 \cdot [\text{URL есть}] + w_3}, \quad S_{se} = \min\left(\frac{k}{3}, 1\right)
```

где k — число сработавших групп социальной инженерии; начальные веса w1 = 0,5, w2 = 0,3, w3 = 0,2. Если URL нет, слагаемое w2 исключается из числителя и знаменателя.

**Функциональные требования:**

1. Правила усиления (override), применяются после формулы:
   - группа `CODE_REQUEST` сработала → Risk Score не ниже 70;
   - P_sms ⩾ 0,8 и P_url,max ⩾ t2 → Risk Score не ниже 85;
   - URL с IP-адресом или Punycode-маскировкой под бренд → Risk Score не ниже 60.
2. Уровни: **низкий** — 0–34, **средний** — 35–69, **высокий** — 70–100. Категория: низкий → `SAFE`, средний → `SUSPICIOUS`, высокий → `FRAUDULENT`.
3. Все веса, пороги и правила хранятся в `risk_config.yaml`.

4. Калибровка: на валидационной выборке SMS и на собственном наборе (задание 3) подобрать веса и границы уровней, чтобы Recall класса «мошенничество» по уровню «средний и выше» был ⩾ 0,95, а доля безопасных SMS с уровнем «высокий» — ⩽ 3%. Результат калибровки — таблица «вариант весов — Recall — доля ложных высоких».
5. Объяснение: каждое срабатывание (вклад модели > 0,5, группа словаря, флаг URL) даёт строку причины на русском языке; причины сортируются по вкладу; выводится не более 6 причин.
6. Рекомендации по уровню, например для высокого: «Не переходите по ссылке, не сообщайте коды и данные карты. Свяжитесь с банком по номеру с обратной стороны карты».
7. Не менее 10 модульных тестов на граничные значения (34/35, 69/70, отсутствие URL, все признаки сразу).

**Ожидаемый результат:** алгоритм Risk Score, конфигурационный файл, таблица калибровки, генератор объяснений.

**Критерии проверки:**

- [ ] Формула и правила совпадают с реализацией и описаны в отчёте.
- [ ] Пороги подобраны на валидации, а не на тесте.
- [ ] Каждый результат содержит ⩾ 1 причины и рекомендацию.
- [ ] Изменение `risk_config.yaml` меняет результат без правки кода.

## Задание 14. Разработка backend и API

**Цель.** Предоставить функциональность анализа через защищённые HTTP-эндпоинты.

**Описание.** Студент реализует REST API из раздела 4.4 на Django REST Framework (или FastAPI).

**Функциональные требования:**

1. Сериализаторы с валидацией: длина текста 1–1000, URL до 2048 символов, удаление управляющих символов.
2. Эндпоинты регистрации, входа, выхода (модуль 2.1, требования ФТ-1.1–ФТ-1.8).
3. Эндпоинты `check/sms`, `check/url`: вызов сервиса из задания 12, расчёт риска, сохранение результата, ответ JSON.
4. Эндпоинты истории и подробного результата с фильтрами и пагинацией; выборка всегда ограничена `user=request.user`.
5. Ограничение частоты (`throttling`): 30 проверок в минуту на пользователя.

6. Единый обработчик ошибок; при отсутствии моделей — ответ `503` с понятным сообщением.
7. Логирование: время обработки запроса, id проверки, категория; текст SMS в лог не пишется.
8. Документация API: Swagger/OpenAPI ( `drf-spectacular` ) или таблица в README с примерами `curl` .
9. Тесты API (не менее 15): успешные сценарии, пустой ввод, слишком длинный текст, неавторизованный доступ, доступ к чужой записи, превышение лимита.

**Ожидаемый результат:** работающие API-эндпоинты с проверкой входных данных и обработкой ошибок.

**Критерии проверки:**

- [ ] Все эндпоинты из раздела 4.4 отвечают с корректными кодами.
- [ ] Запрос чужой записи возвращает `404` .
- [ ] SQL-инъекция и XSS в полях ввода не срабатывают.
- [ ] Тесты API проходят.

## Задание 15. Разработка пользовательского интерфейса

**Цель.** Дать пользователю удобный способ проверить SMS и URL и понять результат.

**Описание.** Студент создаёт страницы из раздела 8.1 на шаблонах Django + Bootstrap 5; проверка выполняется через `fetch` к API без перезагрузки страницы.

**Функциональные требования:**

1. Страница проверки SMS: многострочное поле, счётчик символов (n/1000), кнопка «Проверить сообщение», кнопка «Очистить», 3 кнопки-примера (безопасное, подозрительное, мошенническое), загружающие искусственные тексты.
2. Страница проверки URL: поле ввода, кнопка «Проверить ссылку», подсказка о том, что ссылка не открывается.
3. Во время анализа — индикатор загрузки, кнопка блокируется (защита от двойной отправки).
4. Блок результата: цветной бейдж категории (зелёный / жёлтый / красный), шкала Risk Score, уровень, вероятности моделей, список признаков, объяснение, рекомендация, предупреждение о вероятностном характере.

5. Найденные URL отображаются **неактивным текстом** с выделенным доменом, без гиперссылки.
6. Вывод пользовательского текста только через `textContent` (не `innerHTML`).
7. Цвет не единственный носитель смысла: у каждого бейджа есть текстовая подпись и иконка.
8. Адаптивность при 360, 768, 1366 px; контраст текста не ниже 4,5:1.
9. Ошибки (пустой ввод, превышение лимита, недоступность модели) отображаются рядом с полем на русском языке.

**Ожидаемый результат:** интерактивные страницы, позволяющие проверить SMS и URL.

**Критерии проверки:**

- [ ] Проверка работает без перезагрузки страницы.
- [ ] Текст `<script>alert(1)</script>` отображается как текст.
- [ ] Ссылки в результате не кликабельны.
- [ ] Интерфейс корректен на трёх ширинах экрана (скриншоты в отчёте).

## Задание 16. База данных и история проверок

**Цель.** Сохранять результаты и предоставить пользователю персональную историю.

**Описание.** Студент реализует модели Django по разделу 4.3, миграции и страницу истории.

**Функциональные требования:**

1. Модели `UserProfile`, `Check`, `UrlResult`, `DetectedFeature`, `ModelVersion`; миграции применяются с нуля без ошибок.
2. ER-диаграмма, соответствующая фактическим моделям.
3. Сохранение результата каждой проверки в транзакции (`transaction.atomic`).
4. Маскирование: без согласия пользователя `input_text = NULL`, в `input_masked` номера телефонов, карт (13–19 цифр) и суммы заменены на `***`.
5. Страница «История»: таблица (дата, тип, фрагмент до 60 символов, категория, уровень, Risk Score), пагинация по 20, фильтры по датам, типу, категории, уровню; состояние фильтров сохраняется в адресной строке.
6. Просмотр подробного результата и удаление записи с подтверждением.
7. Профиль: смена пароля, флажок согласия на хранение текстов, кнопка «Удалить всю мою историю».

8. Скрипт `python manage.py seed_demo` создаёт демонстрационного пользователя и 30 проверок из искусственных сообщений.

**Ожидаемый результат:** работающая база данных и персональная история проверок.

**Критерии проверки:**

- [ ] Фильтры дают корректную выборку (проверка на демо-данных).
- [ ] Без согласия полный текст SMS не сохраняется.
- [ ] Пользователь не видит и не может удалить чужие записи.

## Задание 17. Панель мониторинга и визуализация

**Цель.** Показать пользователю и администратору сводную картину проверок.

**Описание.** Студент реализует Dashboard на Chart.js или Plotly.js с данными из `/api/v1/stats/summary/`.

**Функциональные требования:**

1. Карточки: всего SMS, всего URL, безопасных, подозрительных, мошеннических, средний Risk Score.
2. Круговая (кольцевая) диаграмма распределения категорий.
3. Линейный график числа проверок по дням за 30 дней с разбивкой по категориям.
4. Столбчатая диаграмма топ-5 сработавших признаков социальной инженерии.
5. Таблица последних 10 проверок со ссылкой на подробный результат.
6. Переключатель периода: 7 / 30 / 90 дней.
7. Агрегация выполняется в БД (`annotate`, `Count`, `TruncDate`), а не в Python-цикле.
8. Для администратора — те же графики по всем пользователям и блок «Активные версии моделей» с метриками из метаданных.
9. Пустое состояние: при отсутствии проверок — подсказка и кнопка перехода к проверке.
10. Dashboard обновляется сразу после новой проверки (при переходе на страницу).

**Ожидаемый результат:** интерактивная панель мониторинга.

**Критерии проверки:**

- [ ] Числа на карточках совпадают с выборкой из БД.

- [ ] Графики имеют заголовки, подписи осей и легенду.
- [ ] Загрузка при 1000 записях — не более 2 с (НФТ-2).

### Задание 18. Комплексное тестирование и демонстрация

**Цель.** Подтвердить работоспособность, качество и безопасность системы и подготовить защиту.

**Описание.** Студент проводит функциональное, модульное, интеграционное тестирование и тестирование безопасности, оформляет отчёт и демонстрационные сценарии.

**Функциональные требования:**

1. Матрица трассировки: каждое ФТ и НФТ → тест-кейс → результат (пройден / не пройден).
2. Не менее 40 тест-кейсов в формате: id, предусловие, шаги, входные данные, ожидаемый результат, фактический результат.
3. Автотесты `pytest` проходят полностью; покрытие `ml/` и `risk` ⩾ 60%.
4. Тестирование безопасности: SQL-инъекция, XSS, CSRF (запрос без токена), доступ к чужим данным, перебор паролей, проверка заголовков cookie.
5. Тестирование надёжности: пустой ввод, 1001 символ, бинарные данные, URL без домена, недоступность внешнего API, удалённый файл модели.
6. Итоговые метрики обеих моделей на тестовых выборках, матрицы ошибок, метрики на собственном русско-казахском наборе.
7. Анализ ложноположительных и ложноотрицательных результатов (не менее 5 примеров каждого) и предложения по улучшению.
8. Не менее 10 демонстрационных сообщений (раздел 8.3) и пять обязательных сценариев:
   a. Проверка обычного безопасного SMS.
   b. Проверка мошеннического SMS с требованием перевести деньги.
   c. Проверка сообщения с подозрительной ссылкой.
   d. Проверка URL без текста SMS.
   e. Проверка сообщения без ссылки, но с признаками социальной инженерии.
9. Видеозапись демонстрации (3–5 минут) — резервный вариант на случай технических проблем при защите.

**Ожидаемый результат:** отчёт о тестировании, итоговые метрики ML-моделей, работающий проект и демонстрационные сценарии.

**Критерии проверки:**

- [ ] Все 5 сценариев воспроизводятся на свежей установке по README.
- [ ] Отчёт фиксирует фактические, а не ожидаемые результаты, включая неудачные тесты.
- [ ] Результаты демонстрации формируются моделями в реальном времени, а не заготовлены.

## 8. Интерфейс, демонстрация, документация и оценка

Интерфейс выполняется в стиле современной системы информационной безопасности: тёмная или светлая тема, сдержанная палитра, акцентные цвета только для категорий риска.

### 8.1. Страницы приложения

| № | Страница | Адрес | Основное содержимое |
|---|---|---|---|
| 1 | Вход и регистрация | `/login/`, `/register/` | Формы, сообщения об ошибках, ссылка на восстановление пароля (по желанию) |
| 2 | Dashboard | `/` | Карточки, диаграммы, последние проверки |
| 3 | Проверка SMS | `/check/sms/` | Поле ввода, примеры, кнопка «Проверить сообщение» |
| 4 | Проверка URL | `/check/url/` | Поле ввода, кнопка «Проверить ссылку» |
| 5 | Результат анализа | `/checks/<id>/` | Полный разбор результата |
| 6 | История | `/history/` | Таблица, фильтры, пагинация |
| 7 | Статистика | `/stats/` | Расширенные графики, период, данные по уровням риска |
| 8 | Профиль | `/profile/` | Данные, пароль, согласие на хранение текстов, удаление истории |

На странице результата обязательно отображаются:

- исходный текст SMS (или маскированный) либо проверяемый URL неактивным текстом;

- итоговая категория и уровень риска;
- Risk Score и вероятности моделей ( `P_sms` , `P_url` );
- обнаруженные подозрительные признаки;
- объяснение, почему присвоен такой результат;
- рекомендация пользователю;
- предупреждение о вероятностном характере оценки и дата проверки.

Пример отображения (демонстрационный макет, а не заранее заданный результат модели):

> **Результат проверки: МОШЕННИЧЕСКОЕ СООБЩЕНИЕ** · Уровень риска: высокий · Risk Score: 87/100
>
> Обнаруженные признаки: сообщение требует срочного действия; присутствует требование перейти по ссылке; упоминается блокировка банковского аккаунта; URL имеет признаки подозрительной структуры (домен похож на название банка, подозрительное слово `verify` ).
>
> Рекомендация: не переходите по ссылке и не вводите персональные или банковские данные. Свяжитесь с банком по официальному номеру.

## 8.2. Сценарий демонстрации на защите

1. Открыть веб-приложение, выполнить вход под демонстрационным пользователем.
2. Перейти на страницу проверки SMS, вставить текст сообщения.
3. Нажать «Проверить сообщение».
4. Система выполняет анализ текста (показать индикатор загрузки).
5. Если в тексте есть ссылка — показать, что она проанализирована отдельно (блок «Найденные ссылки»).
6. Показать категорию, уровень риска и Risk Score.
7. Открыть подробное объяснение и признаки.
8. Перейти в «Историю» — новая запись присутствует.
9. Открыть Dashboard — счётчики и графики обновились.
10. Проверить отдельный URL на странице проверки ссылок.
11. Дополнительно: показать ввод текста, которого нет в демонстрационном наборе (предлагает комиссия), чтобы подтвердить работу моделей в реальном времени.

## 8.3. Демонстрационные сообщения

Все сообщения искусственные. Ссылки используют только зарезервированные домены ( `.example` , `.test` , `example.com` ), которые не ведут на реальные ресурсы. Колонка «Ожидание» — гипотеза автора набора, а не гарантированный ответ модели; фактический результат фиксируется в отчёте.

| № | Язык | Текст | Ожидание |
|---|---|---|---|
| 1 | ru | Привет! Встречаемся завтра в 18:00 у входа в библиотеку. | Безопасное |
| 2 | en | Your order #4821 has been shipped and will arrive on Monday. | Безопасное |
| 3 | kk | Сәлем! Ертең сағат 10:00-де кездесеміз бе? | Безопасное (с предупреждением о языке) |
| 4 | ru | Ваш код подтверждения: 4821. Никому не сообщайте этот код. | Безопасное или подозрительное (проверка ложных срабатываний) |
| 5 | ru | Ваша карта заблокирована! Срочно подтвердите данные по ссылке: http://secure-bank-verify.example/login | Мошенническое |
| 6 | en | URGENT: Your account will be suspended in 24 hours. Verify now at http://192.0.2.15/account-update | Мошенническое |
| 7 | ru | Мама, у меня проблемы, срочно переведи 50 000 на этот номер, потом всё объясню. Никому не говори. | Мошенническое (без ссылки — сценарий 5) |
| 8 | en | Congratulations! You won a $1000 gift card. Claim your prize: bit.ly/prize-claim-test | Мошенническое |
| 9 | ru | Сотрудник службы безопасности банка: назовите код из SMS для отмены подозрительной операции. | Мошенническое (без ссылки, `CODE_REQUEST` ) |
| 10 | kk | Картаңыз бұғатталды. Жедел түрде мына сілтеме арқылы растаңыз: http://bank-kz-verify.test | Подозрительное или мошенническое |

| № | Язык | Текст | Ожидание |
|---|---|---|---|
| 11 | ru | Ваша посылка ожидает оплаты таможенного сбора 350 ₸: http://post-delivery-pay.example | Подозрительное или мошенническое |
| 12 | URL | https://www.example.com/about | Безопасная ссылка (сценарий 4) |
| 13 | URL | http://xn--exmple-cua.test/secure/login?verify=1 | Опасная ссылка (Punycode + ключевые слова) |

## 8.4. Итоговая документация

Пояснительная записка (40–60 страниц) включает:

1. Описание актуальности проекта.
2. Цель и задачи разработки.
3. Анализ предметной области.
4. Анализ существующих аналогов.
5. Функциональные и нефункциональные требования.
6. Архитектуру системы.
7. Диаграмму вариантов использования (Use Case).
8. Диаграмму деятельности (Activity Diagram) — процесс проверки SMS.
9. Диаграмму последовательности (Sequence Diagram) — от нажатия кнопки до сохранения результата.
10. ER-диаграмму базы данных.
11. Описание наборов данных (паспорта).
12. Описание алгоритмов машинного обучения.
13. Результаты обучения и сравнения моделей.
14. Описание реализации модулей.
15. Скриншоты работающего приложения.
16. Результаты функционального тестирования.
17. Выводы и направления дальнейшего развития (трансформерные модели для казахского языка, мобильное приложение, расширение наборов данных).

Приложения: ссылка на репозиторий, README, отчёт о тестировании, ноутбуки, видеозапись демонстрации.

## 8.5. Критерии оценки

| Критерий | Баллы | Что проверяется |
|---|---|---|
| Анализ предметной области и проектирование | 10 | Задания 1–2, диаграммы, требования |
| Подготовка и обработка данных | 10 | Задания 3–5, 10, отсутствие утечки данных |
| Реализация SMS-классификатора | 15 | Задания 6–8, сравнение ⩾ 3 алгоритмов, метрики |
| Реализация URL-классификатора | 15 | Задания 9–11, признаки, сравнение ⩾ 2 алгоритмов |
| Совместный анализ и оценка риска | 10 | Задания 12–13, калибровка, объяснения |
| Backend и база данных | 10 | Задания 14, 16, безопасность API |
| Пользовательский интерфейс и Dashboard | 10 | Задания 15, 17 |
| Тестирование и оценка качества моделей | 10 | Задание 18, матрица трассировки, анализ ошибок |
| Демонстрация и защита проекта | 10 | Сценарии 8.2, ответы на вопросы |
| **Итого** | **100** | |

Соответствие оценке: 90–100 — «отлично», 75–89 — «хорошо», 60–74 — «удовлетворительно», менее 60 — «неудовлетворительно» (шкала уточняется кафедрой).

## 8.6. Ограничения и дополнительные условия

- Все 18 заданий выполняются последовательно; каждое подтверждается коммитом и артефактом.
- Запрещено приложение, которое отображает заранее подготовленные результаты: классификация выполняется обученными моделями по реальному вводу.
- SMS-модель и URL-модель обучаются отдельно, сохраняются в файлы и загружаются приложением.
- Внешние API репутации URL допускаются только как дополнительный источник и не заменяют собственный ML-модуль.

- Система учитывает ложноположительные и ложноотрицательные результаты и не утверждает, что ссылка гарантированно безопасна.
- Реальные вредоносные ссылки не открываются ни при обучении, ни при тестировании, ни при демонстрации.
- Поддержка казахского и русского языков заявляется только в пределах подтверждённых тестом метрик.
- Используются только открытые наборы данных с указанием лицензии и искусственные сообщения; реальные SMS третьих лиц без согласия не используются.

## 9. Пример итогового продукта (эталонные макеты экранов)

На защите студент демонстрирует **работающее веб-приложение в браузере**, а не Python-код в IDE или Jupyter Notebook. Код и ноутбуки показываются только по запросу комиссии.

Ниже приведены эталонные макеты экранов, к которым должен прийти проект. Состав элементов на каждом экране обязателен; цвета, шрифты и расположение могут отличаться. Все числа и вероятности на макетах условные: на защите они формируются обученными моделями по реальному вводу.

Требования к демонстрации итогового продукта:

- приложение запущено локально или на сервере и открыто в браузере;
- вход выполняется под демонстрационным пользователем;
- каждый экран рис. 1–5 открывается и работает с реальными данными;
- скриншоты всех пяти экранов включаются в пояснительную записку (раздел «Скриншоты работающего приложения»).

## 9.1. Страница входа и регистрации

**Вход в систему: логин, пароль и переход к регистрации**

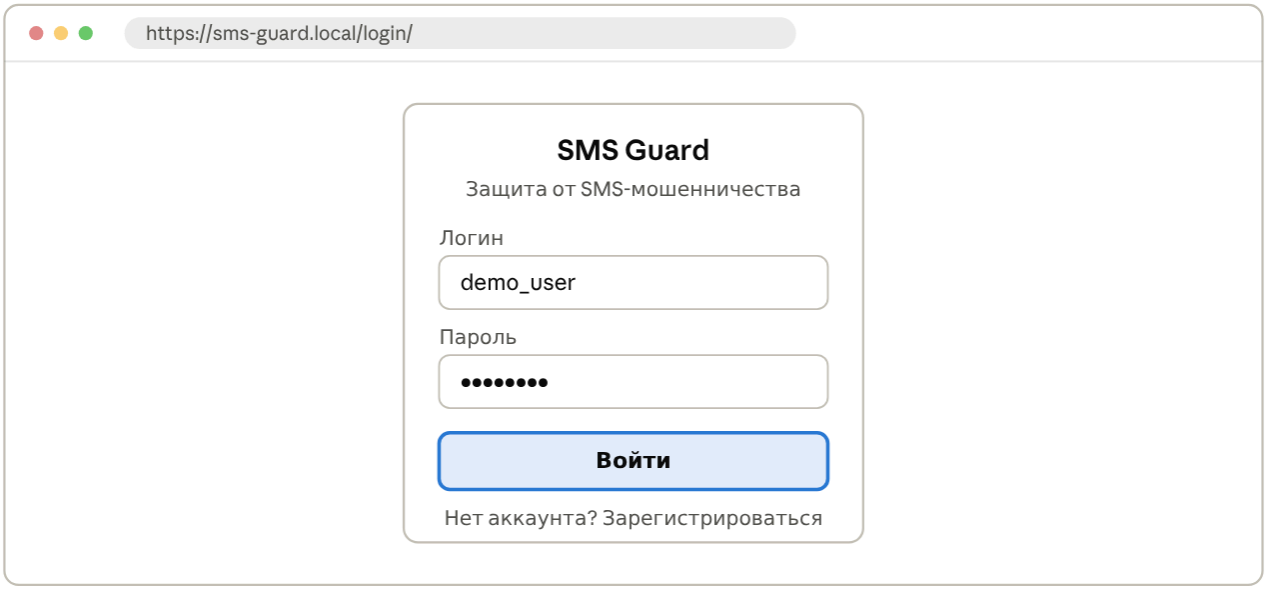



Рис. 1. Страница входа и регистрации

Обязательные элементы: поля логина и пароля, кнопка «Войти», ссылка на регистрацию, сообщение об ошибке при неверных данных. Без входа остальные страницы недоступны.

## 9.2. Главная панель (Dashboard)

**Главная панель: счётчики, диаграммы и последние проверки**

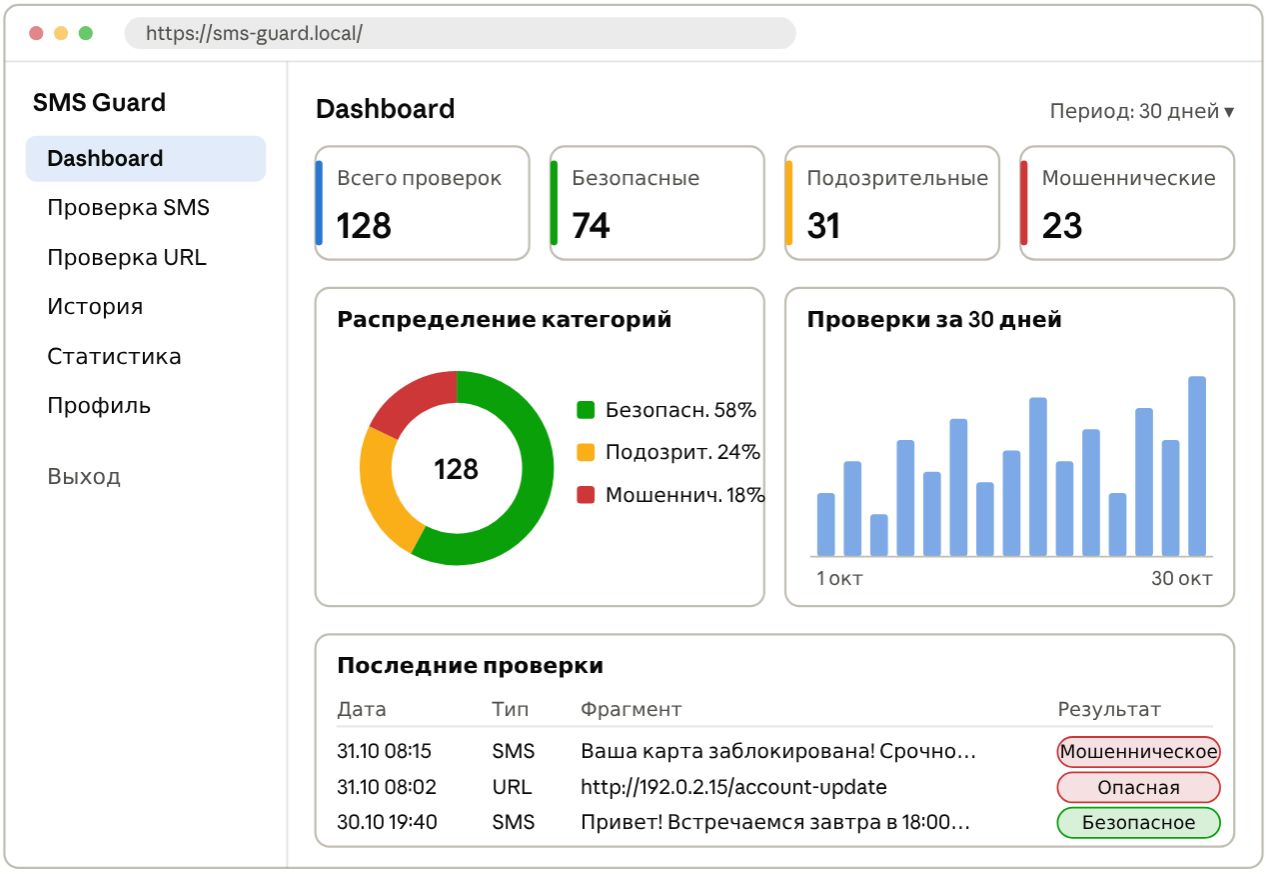



Рис. 2. Главная панель (Dashboard)

Обязательные элементы: боковое меню, четыре карточки-счётчика, диаграмма распределения категорий, график проверок по дням, таблица последних проверок с цветными метками. После новой проверки счётчики и графики обновляются.

## 9.3. Проверка SMS и результат анализа

**Проверка SMS: ввод текста и подробный результат анализа**

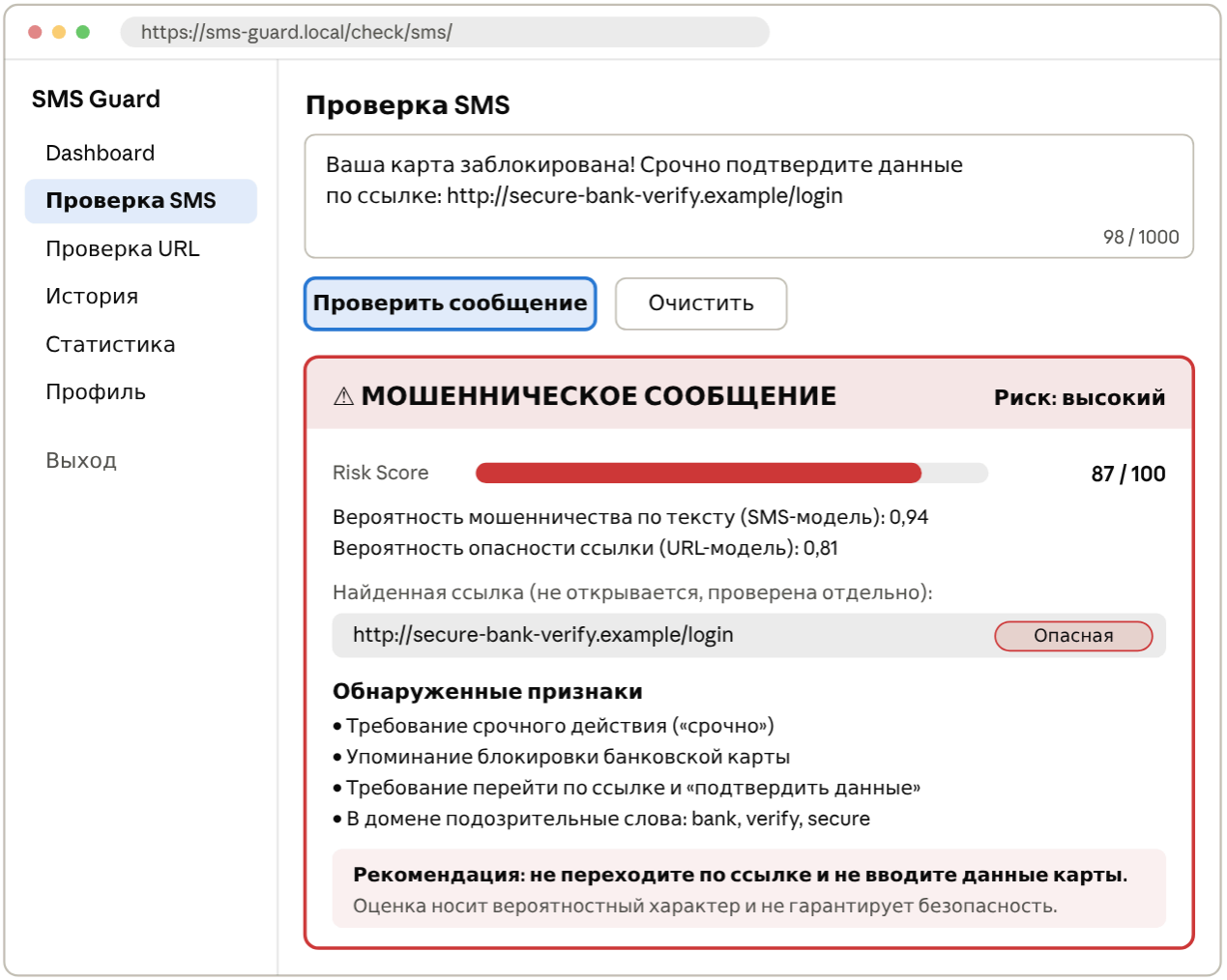



Рис. 3. Проверка SMS и результат анализа

Это главный экран защиты. Обязательные элементы: поле ввода со счётчиком символов, кнопка «Проверить сообщение», цветной заголовок категории, уровень риска и шкала Risk Score, вероятности обеих моделей, найденная ссылка неактивным текстом с отдельной оценкой, список признаков, рекомендация и предупреждение о вероятностном характере оценки.

## 9.4. Проверка отдельной ссылки (URL)

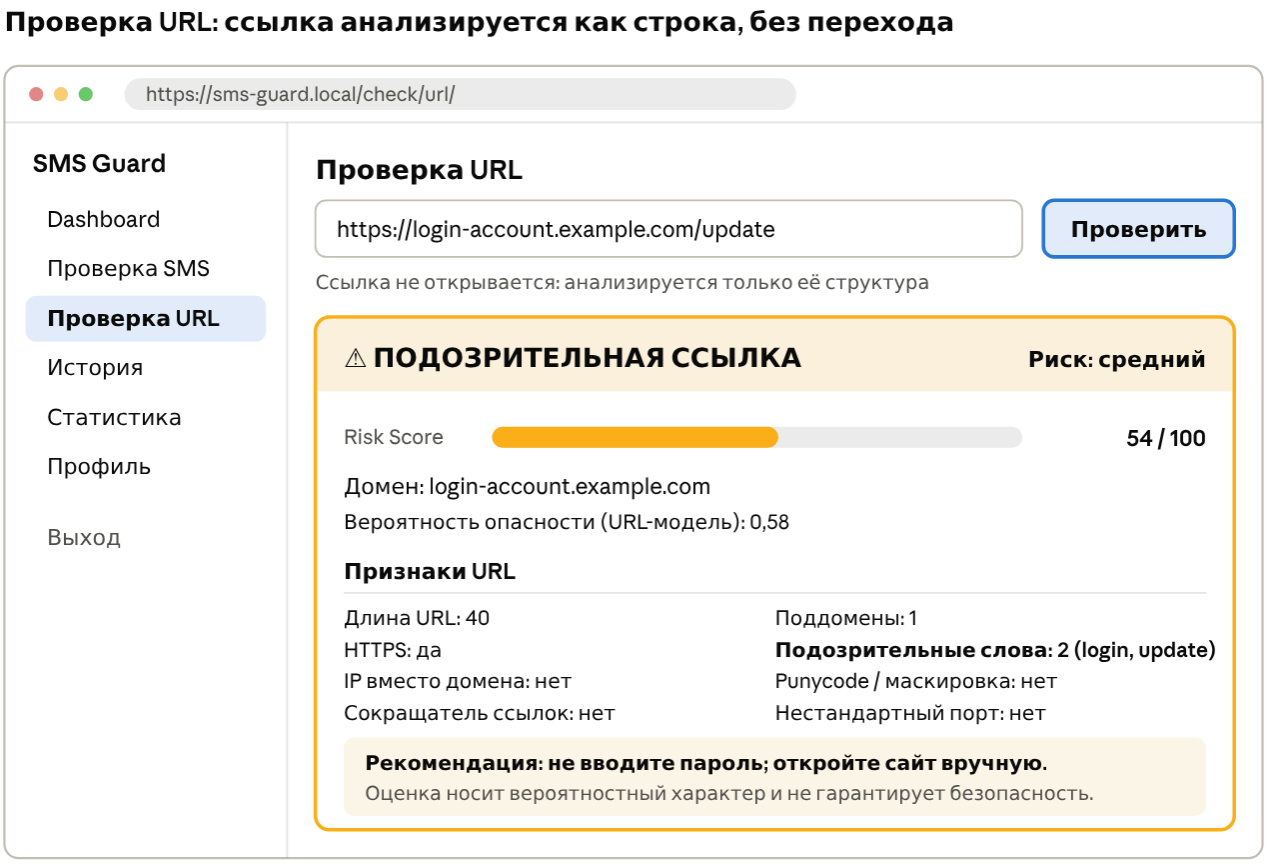



Рис. 4. Проверка отдельной ссылки (URL)

Обязательные элементы: поле ввода URL, подсказка о том, что ссылка не открывается, категория и уровень риска, выделенный домен, вероятность URL-модели и таблица извлечённых признаков, в которой сработавшие признаки выделены.

## 9.5. История проверок

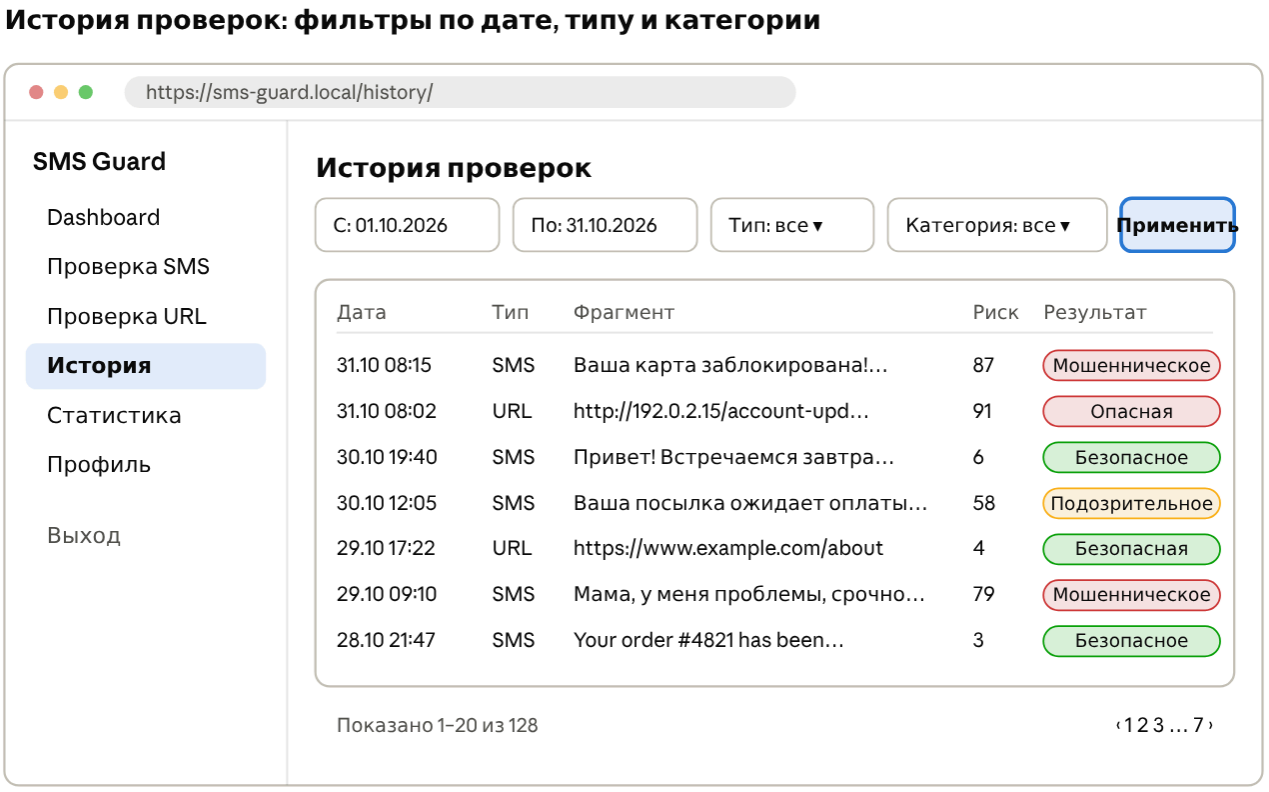



Рис. 5. История проверок

Обязательные элементы: фильтры по датам, типу и категории, таблица с датой, типом, фрагментом, Risk Score и цветной меткой результата, пагинация по 20 записей. По нажатию на строку открывается подробный результат (рис. 3). Пользователь видит только свои записи.